

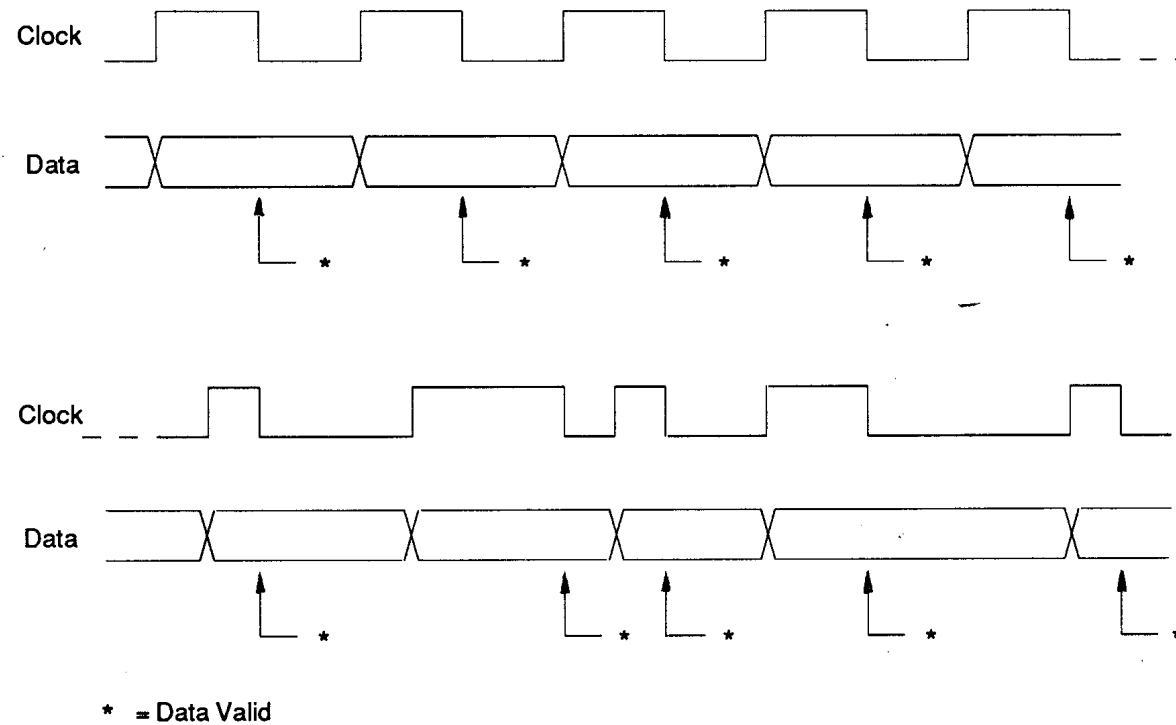
COMUNICAÇÃO SERIAL SÍNCRONA

Paulo Valim

COMUNICAÇÃO SERIAL SÍNCRONA

- **A comunicação serial distribui dos bits a serem transmitidos no tempo e o receptor deve ser capaz de reconhecer quando um bit começa e termina.**
- **Quando um clock é enviado juntamente com os dados, o receptor facilmente delimita o bit porque eles ocorrem simultaneamente com os bits de clock.**
- **O sinal de clock pode ter algumas “formas” diferentes, como o estado alto ou baixo ou a transição entre estados.**
- **Dentro de certos limites, impostos pelo hardware e a aplicação, não se estabelece uma taxa de transmissão fixa. Considerando que o dado esta associado à forma do clock, a taxa de clock não é significativa e não tem que ser constante ao longo da transmissão. Cada bit é sincronizado um-a-um.**

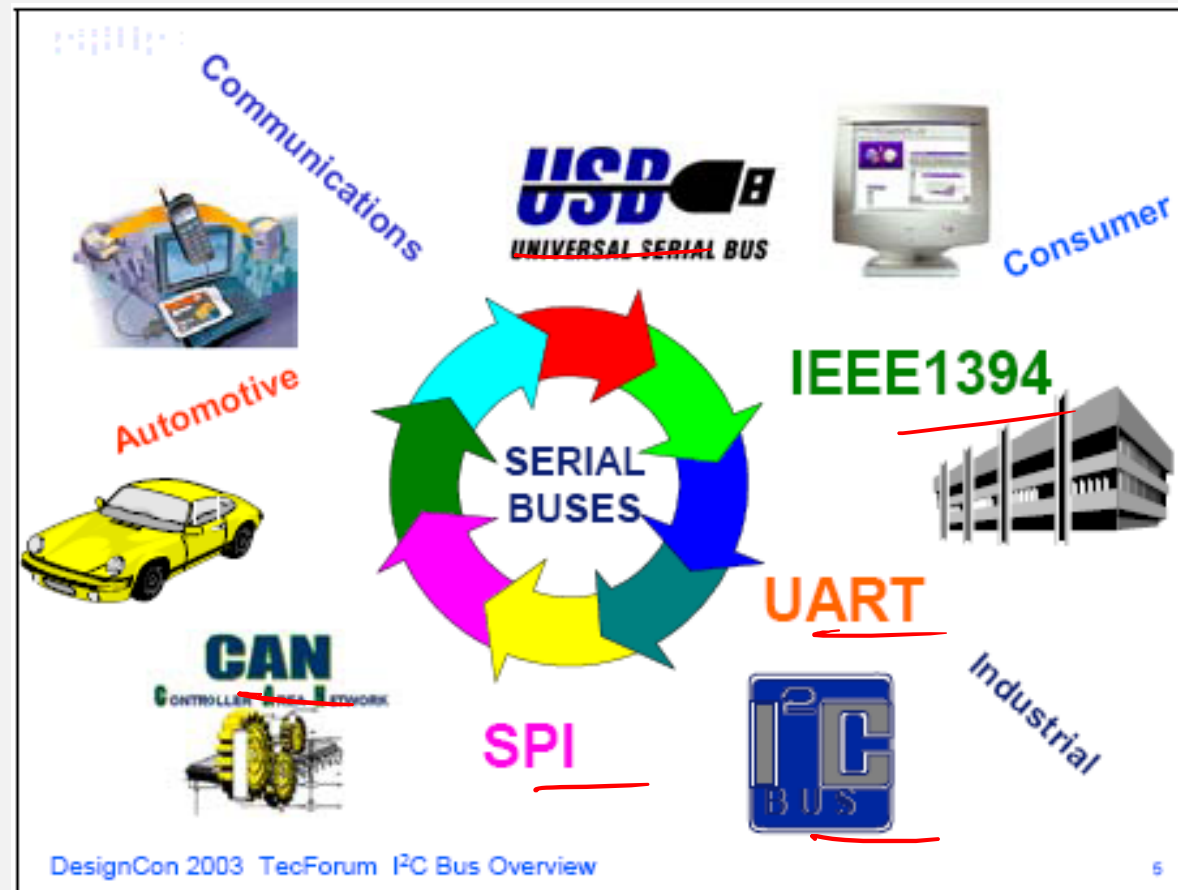
EXEMPLO DE COMUNICAÇÃO SERIAL SÍNCRONA



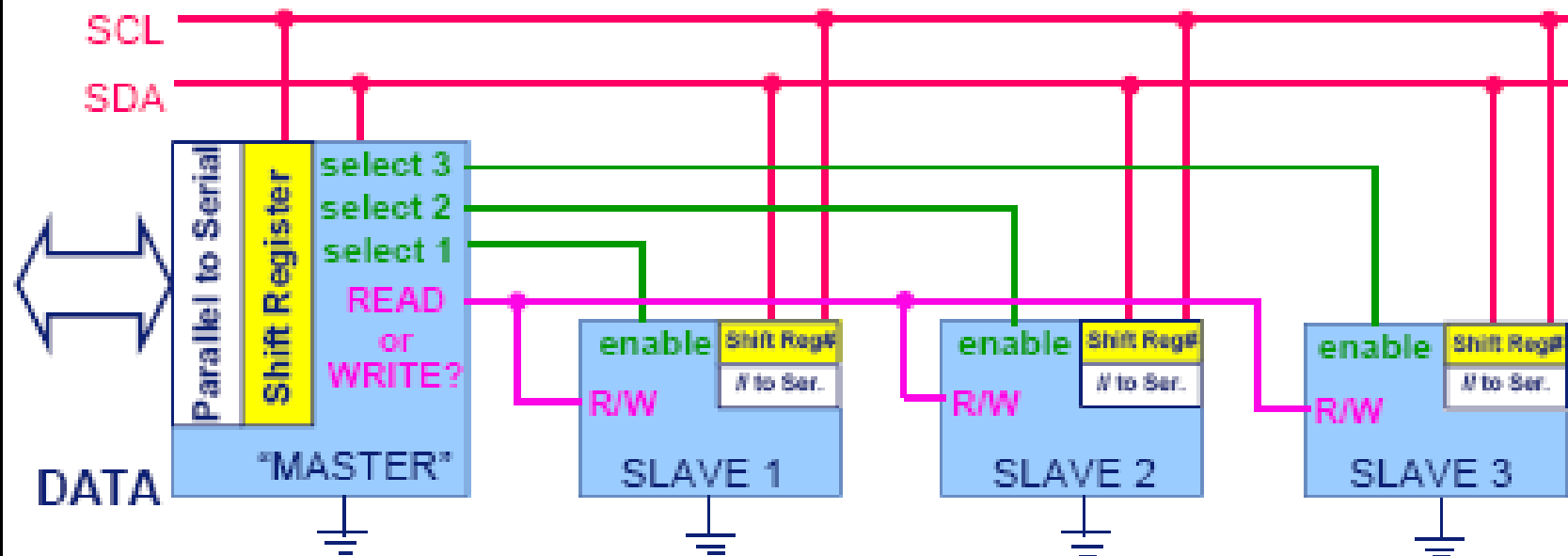
COMUNICAÇÃO SERIAL EM SISTEMAS EMBARCADOS

- **Significativa redução do número de linhas de controle (portas) necessárias para transferir dados para múltiplos periféricos;**
- **O preço da vantagem citada anteriormente são taxas menores de transferência;**

EXEMPLOS DE INTERFACES SERIAIS



General concept for Serial communications



- A point to point communication does not require a Select control signal
- An asynchronous communication does not have a Clock signal
- Data, Select and R/W signals can share the same line, depending on the protocol
- Notice that Slave 1 cannot communicate with Slave 2 or 3 (except via the 'master')
Only the 'master' can start communicating. Slaves can 'only speak when spoken to'

PROTOCOLO SERIAL SÍNCRONO

- A ação dos dispositivos em um barramento serial estão restritas pelo protocolo do barramento.
- O protocolo é o conjunto de regras e procedimentos para assegurar o sucesso na transferência de dados.
- O protocolo define:
 - Formato do bit de dado;
 - Formato do clock e procedimentos que protegem contra todas as possibilidades de perda de dados e bloqueio na comunicação;
 - Somente um dispositivo por vez pode estar no controle do barramento. Este dispositivo é denominado controlador. Os outros dispositivos no barramento são denominados alvos.
 - O controlador inicia e termina uma transferência, endereça o dispositivo alvo e gera o sinal de clock.
 - Alguns protocolos seriais permitem múltiplos controladores no barramento, mas permite que somente um dispositivo por vez assuma o papel de controlador.

DISPOSITIVOS PERIFÉRICOS SERIAIS SÍNCRONOS:

- Dispositivos de hardware com interface serial, geralmente desempenham uma função específica. Alguns exemplos são:
 - Memórias (EEPROM e RAM);
 - Conversores A/D;
 - Conversores D/A;
 - Chaveadores e multiplexadores;
 - Relógios de tempo real;
 - Portas de entrada/saída;
 - Geradores de Tons (incluindo DTMF e tons musicais);
- Quando uma função específica não está disponível como um dispositivo de interface serial, a função do periférico e a interface serial pode ser modelada com um micro-controlador.

6. EXEMPLOS DE DISPOSITIVOS I²C



24AA256
EEPROM



LM75
Sensor de Temperatura



DS3231
RTC



PCF8574
Expansor de I/O

EXEMPLOS DE PADRÕES DE COMUNICAÇÃO SERIAL SÍNCRONA:

- **Vários esquemas de comunicação serial síncrona tem sido desenvolvidos. Os mais usados são:**
 - **I2C BUS (philips);**
 - **SPI BUS (motorola);**
 - **Microwire Interface (National);**

I2C BUS

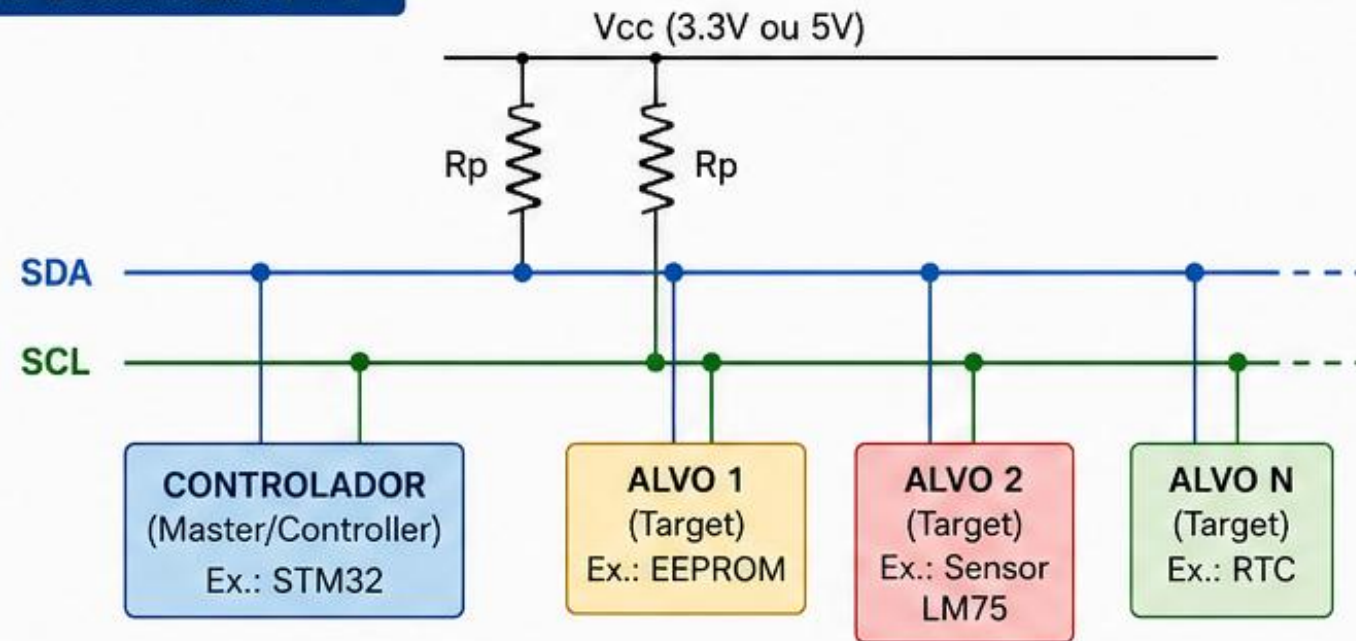
- **I2C (Inter Integrated Circuit)** é um barramento serial de 2 fios para transferência de dados de 8 bits.
- Os 2 fios são: serial clock e serial data.
- O serial data é bi-direcional mas os dados podem fluir somente em um sentido por vez.
- Dispositivos no barramento são definidos como controladores e alvos.
- Um controlador é o dispositivo que inicia a transferência da informação no barramento e gera o sinais de clock e controle.
- Todos os sinais de controle são colocados na linha de dados.
- Cada dispositivo no barramento tem um endereço único chamado endereço alvo. Um dispositivo pode ser acessado somente quando está atuando como alvo.

I2C BUS

- Quando um controlador deseja iniciar a comunicação com um alvo, ele transmite o endereço do alvo e o alvo endereçado responde via uma seqüência de handshake com o controlador.
- Neste momento, somente o controlador e o alvo estão ativos no barramento e o alvo permanece ativo até o controlador o desative.
- Ambas as linhas (clock serial = **SCL** e dado serial = **SDA**) são linhas bidirecionais e devem ser conectadas à **VCC** via resistor de **PULL-UP**;

I2C BUS

1. BARRAMENTO I²C



- SDA: linha de dados (Serial Data)
- SCL: linha de clock (Serial Clock)
- Rp: resistores de pull-UP (ex.: 4,7 kΩ)
- Todos os dispositivos compartilham o mesmo barramento.

I2C

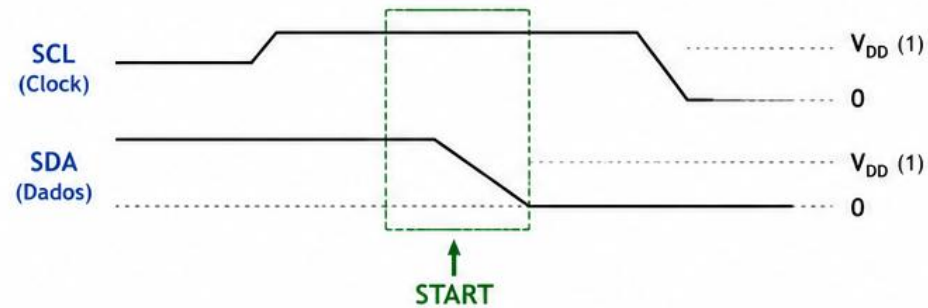
- **Endereço definido em 7 bits, sendo 16 endereços reservados, resultando em 112 nós (existe um limite para a capacitância máxima do barramento: 400 pF);**
- **Trabalha com diversas taxas de transferência (valores de frequência máxima):**
 - 100Kbit/s (standard mode)
 - 10Kbits/s (low-speed mode)
 - 400Kbits/s (fast mode)
 - 1Mbits/s (fast mode plus)
 - 3.4Mbits/s (high speed mode)

I2C BUS – START E STOP

2. CONDIÇÕES DE START E STOP

2.1 CONDIÇÃO START

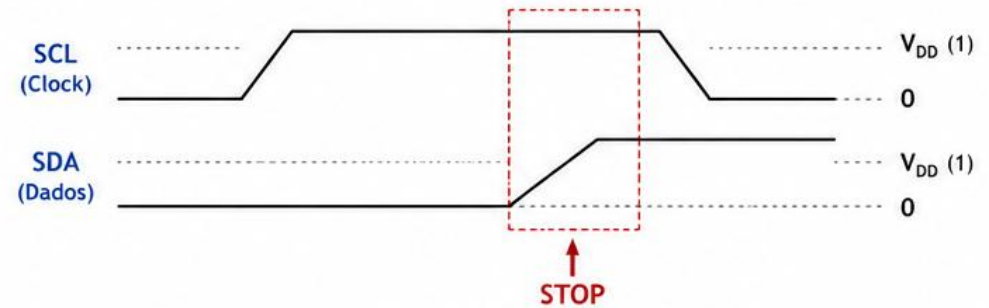
A condição START ocorre quando SDA transiciona de 1 para 0 enquanto SCL está em nível alto.



✓ START: SDA ↓ enquanto SCL = 1

2.2 CONDIÇÃO STOP

A condição STOP ocorre quando SDA transiciona de 0 para 1 enquanto SCL está em nível alto.



✓ STOP: SDA ↑ enquanto SCL = 1

✦ Observações:

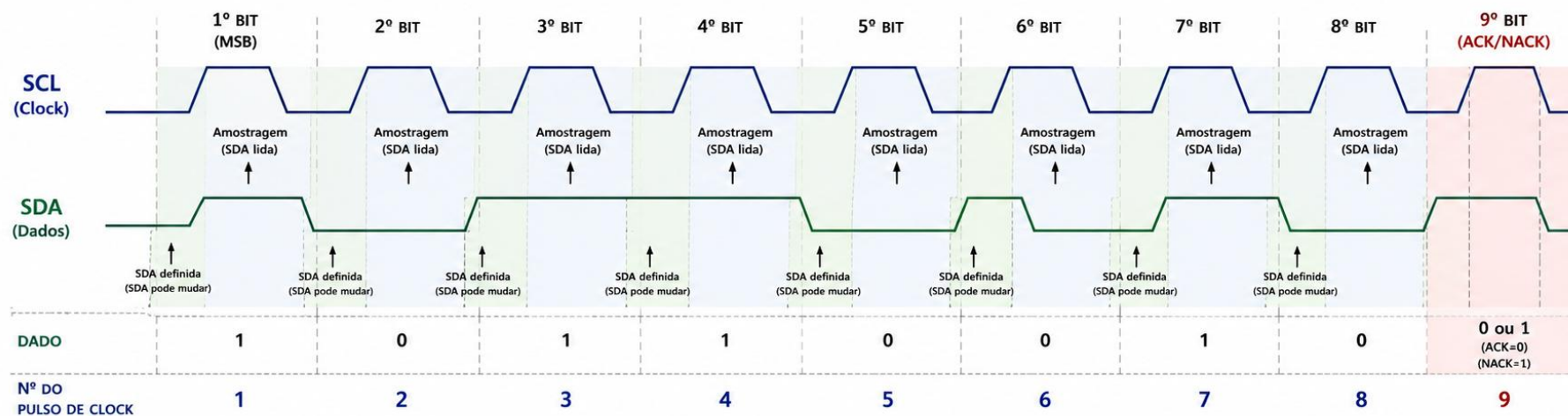
- As linhas SDA e SCL são do tipo “open-drain” (ou “open-collector”) e precisam de resistores de pull-up.
- START e STOP delimitam as transações no barramento I²C.

TRANSMISSÃO DE BITS

TRANSMISSÃO DE 1 BYTE NO I²C (8 BITS DE DADOS + ACK/NACK)

- O dado (SDA) deve ser definido enquanto SCL está em nível baixo.
- O dado é amostrado (lido) quando SCL está em nível alto.
- Após os 8 bits, o receptor envia um bit de ACK (nível baixo) ou NACK (nível alto).
- Dados são transmitidos do bit mais significativo (MSB) para o menos significativo (LSB).

- SCL (Clock do barramento)
- SDA (Dados do barramento)
- SCL em nível alto (dados válidos)
- SCL em nível baixo (SDA pode mudar)
- Bit de ACK/NACK (SCL em nível alto)



REGRAS IMPORTANTES

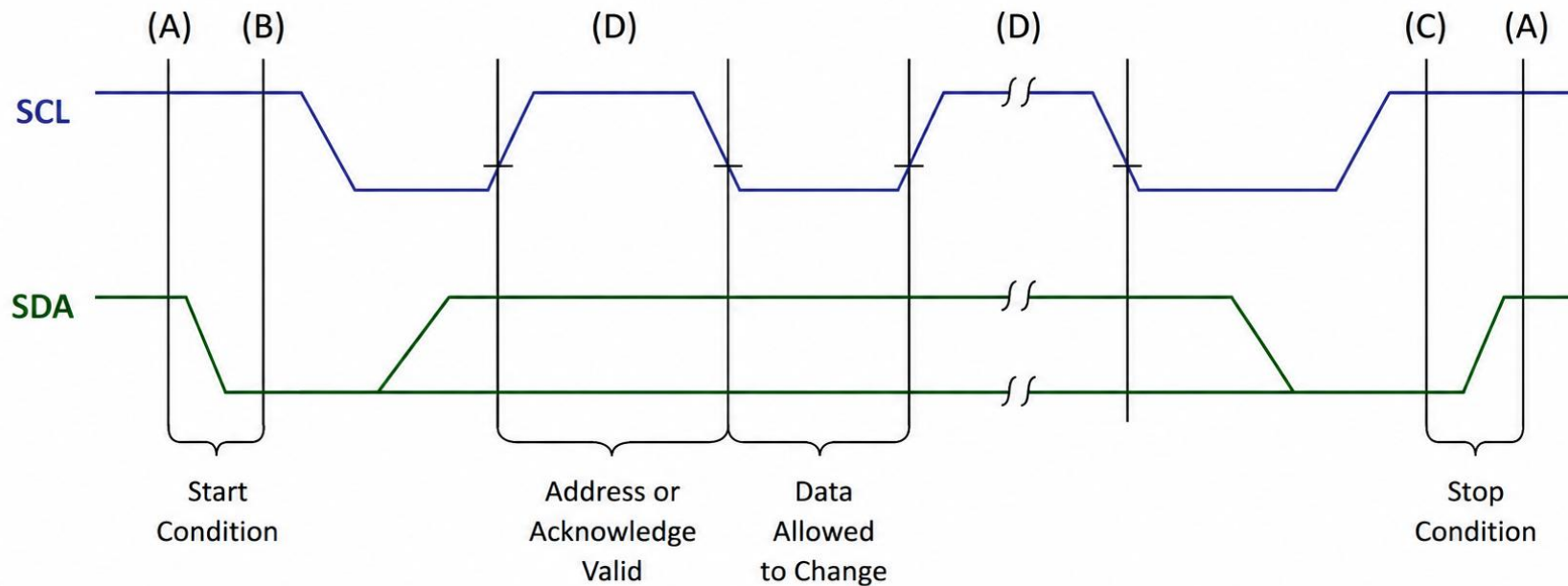
- SDA só pode mudar quando SCL está em nível baixo.
- SDA deve estar estável (definida) antes da borda de subida do clock.
- O receptor amostra o nível de SDA na borda de subida (quando SCL vai de 0 para 1).

ACK / NACK (9º BIT)

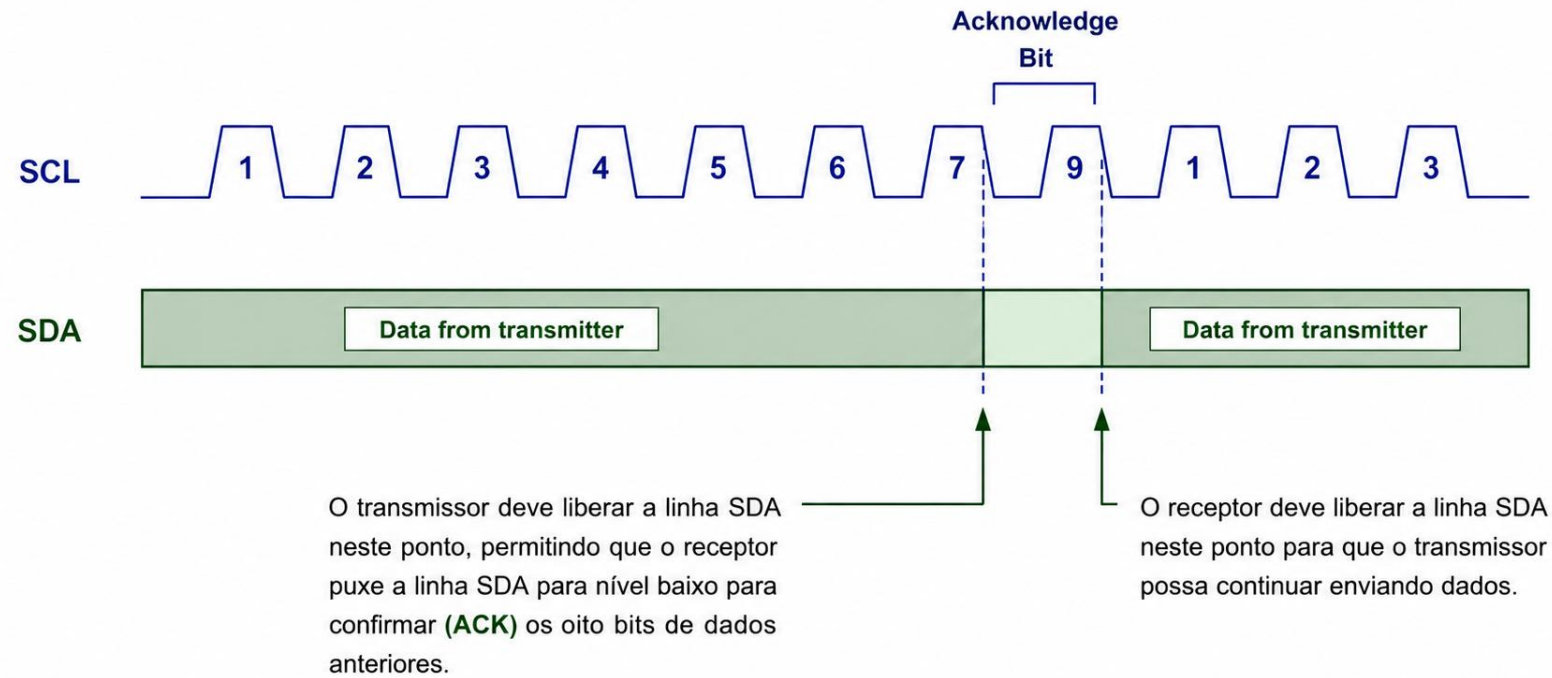
- Após os 8 bits de dados, o receptor coloca SDA em 0 (ACK) para confirmar o recebimento.
- Se o receptor não puder receber mais dados, ele deixa SDA em 1 (NACK).
- O emissor libera SDA (linha sobe com pull-up) durante o 9º pulso de clock.

SDA no 9º bit	Significado
0	ACK (confirmação)
1	NACK (não confirmação)

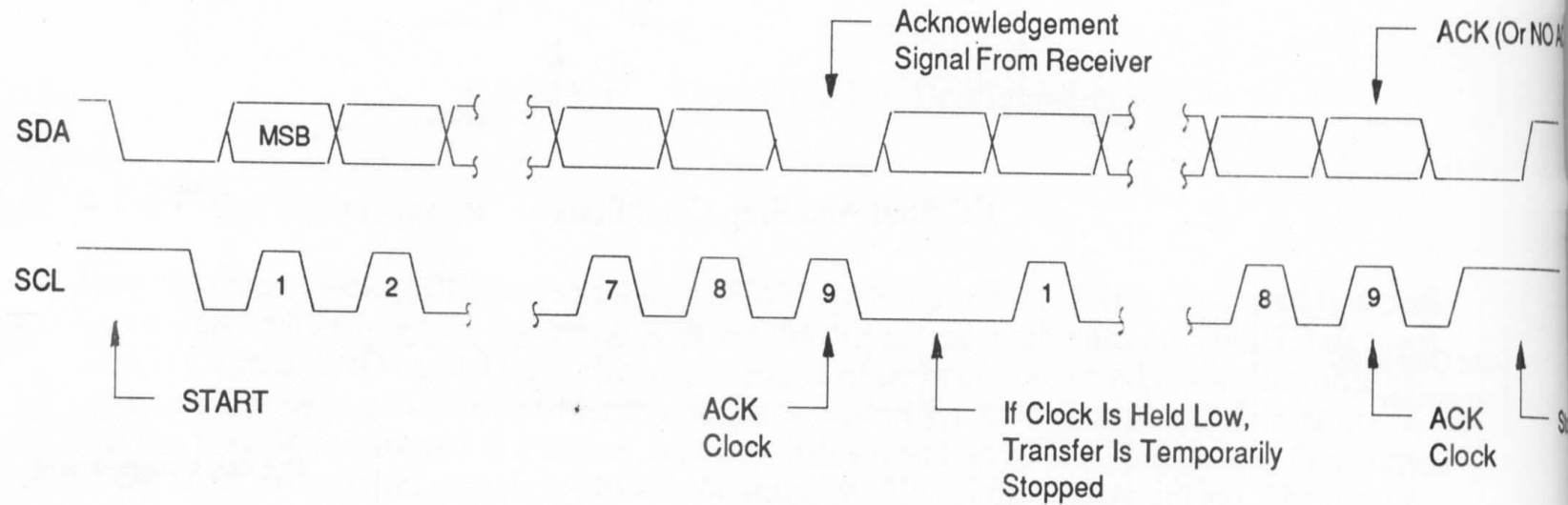
PONTOS IMPORTANTES A SEREM OBSERVADOS



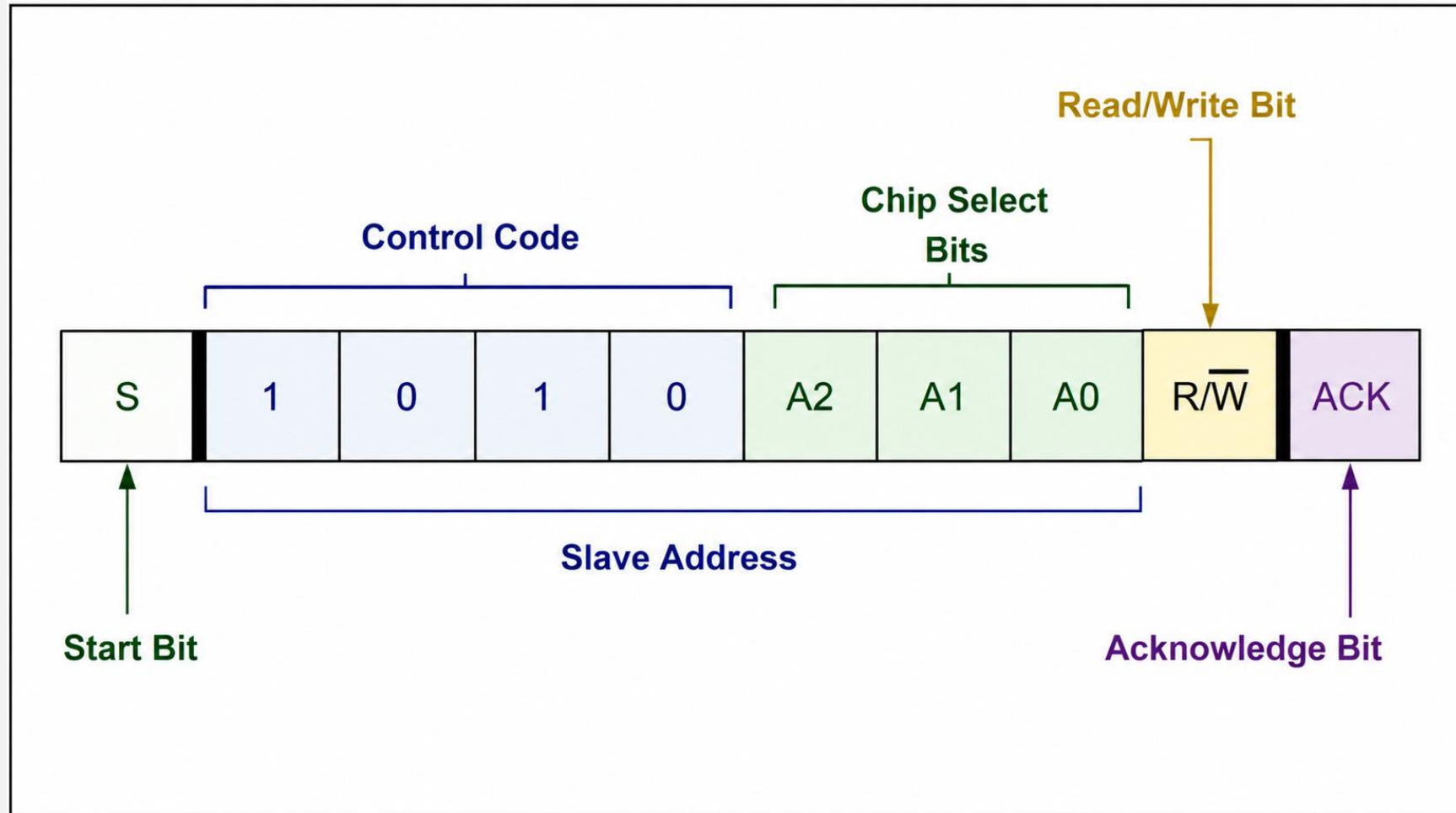
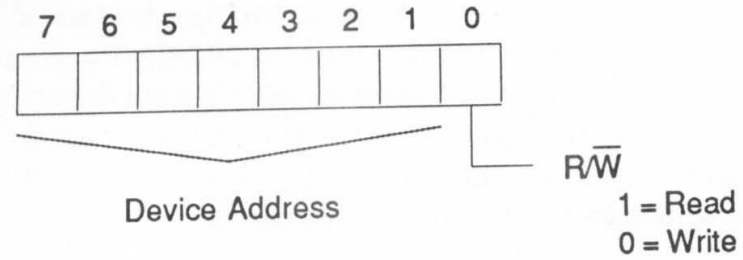
I2C BUS – RECONHECIMENTO



I2C BUS – DATA TRANSFER



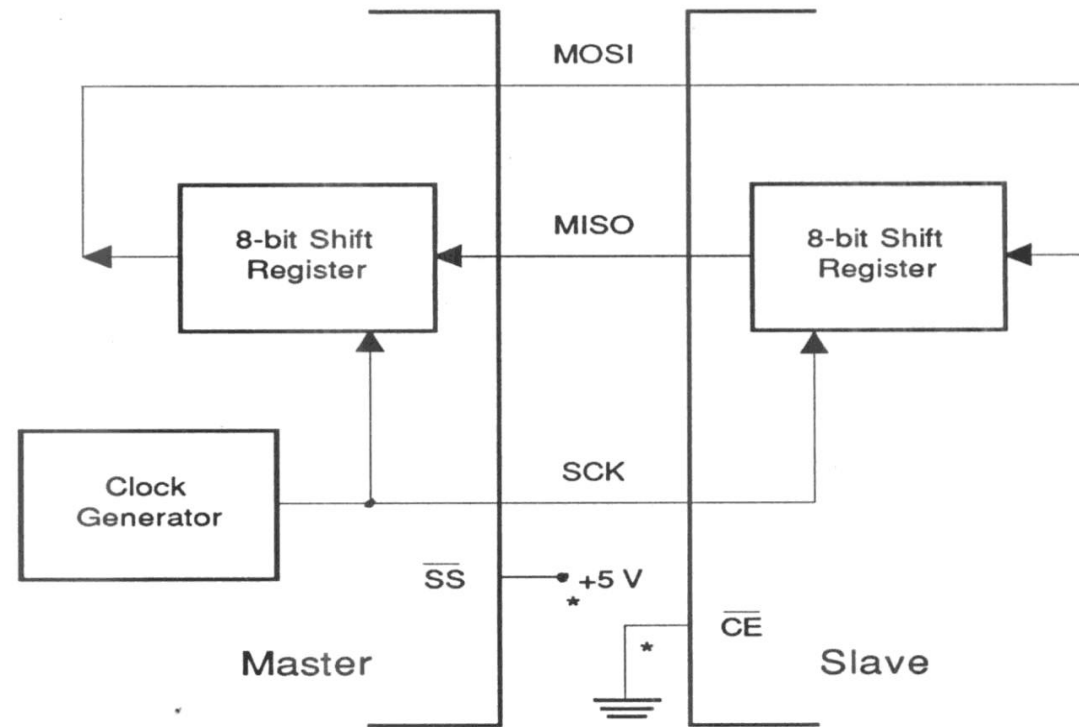
I2C BUS
EEPROM
ADDRESS
READ)



SPI SERIAL BUS

- **SPI (Serial Peripheral Interface)** é um barramento de 3 linhas para aplicações de transferência de 8 bits de dados.
- Duas das três linhas transferem dados (uma linha para cada direção) e a terceira é o sinal de clock.
- Alguns dispositivos podem somente transmitir e outros somente receber. Geralmente o dispositivo que transmite vai também receber. Um exemplo de dispositivo que somente recebe seria um display.
- Dispositivos no barramento são definidos como mestre e escravos. Um mestre é o dispositivo que inicia a transferência no barramento e gera o sinal de clock e de controle. Escravo é o dispositivo controlado pelo mestre.
- Cada escravo no barramento é controlado por uma linha de “chip enable”. O escravo está ativado quanto estiver selecionado

SPI SERIAL BUS



* See section 9.1

Figure 9-1 SPI Bus

SPI SERIAL BUS – ESPECIFICAÇÕES

- **Todas as linhas no barramento SPI são unidirecionais.**
- **O sinal sobre a linha de clock (SCK) é gerado pelo mestre e sincroniza a transferência de dados.**
- **A linha Master-Out, Slave-In (MOSI) leva dados do mestre para o escravo e a linha Master-In, Slave-Out (MISO) leva dados do escravo para o mestre.**
- **Um dispositivo SPI escravo é selecionado por uma lógica zero na sua linha de seleção (chip enable).**

SPI SERIAL BUS

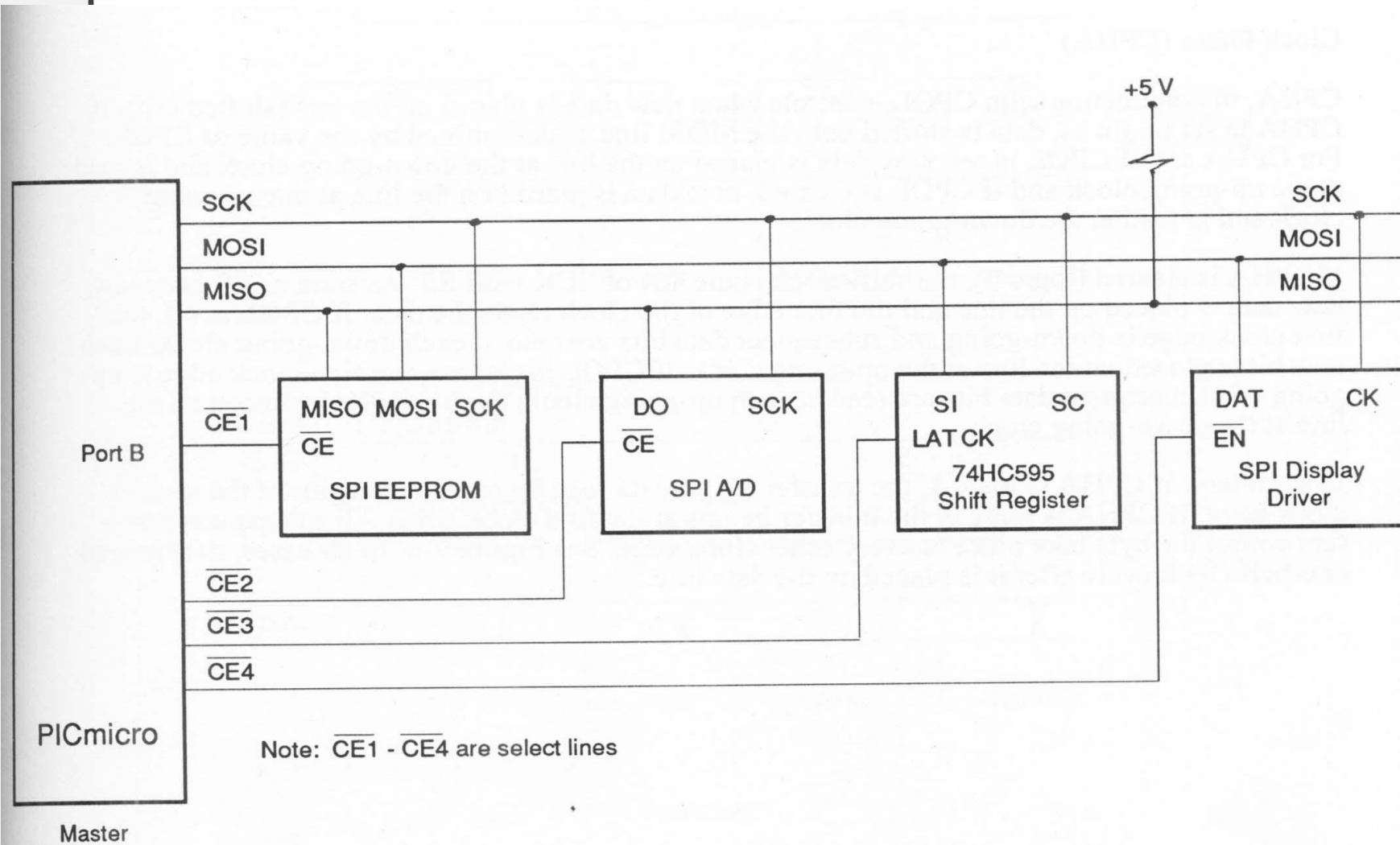
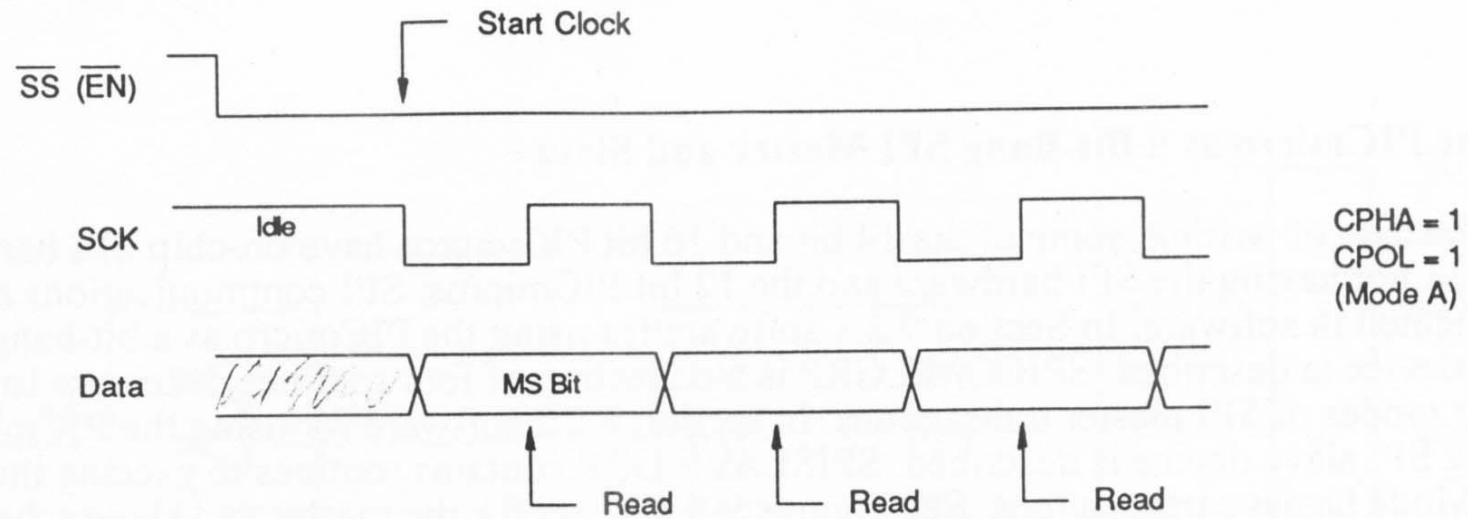
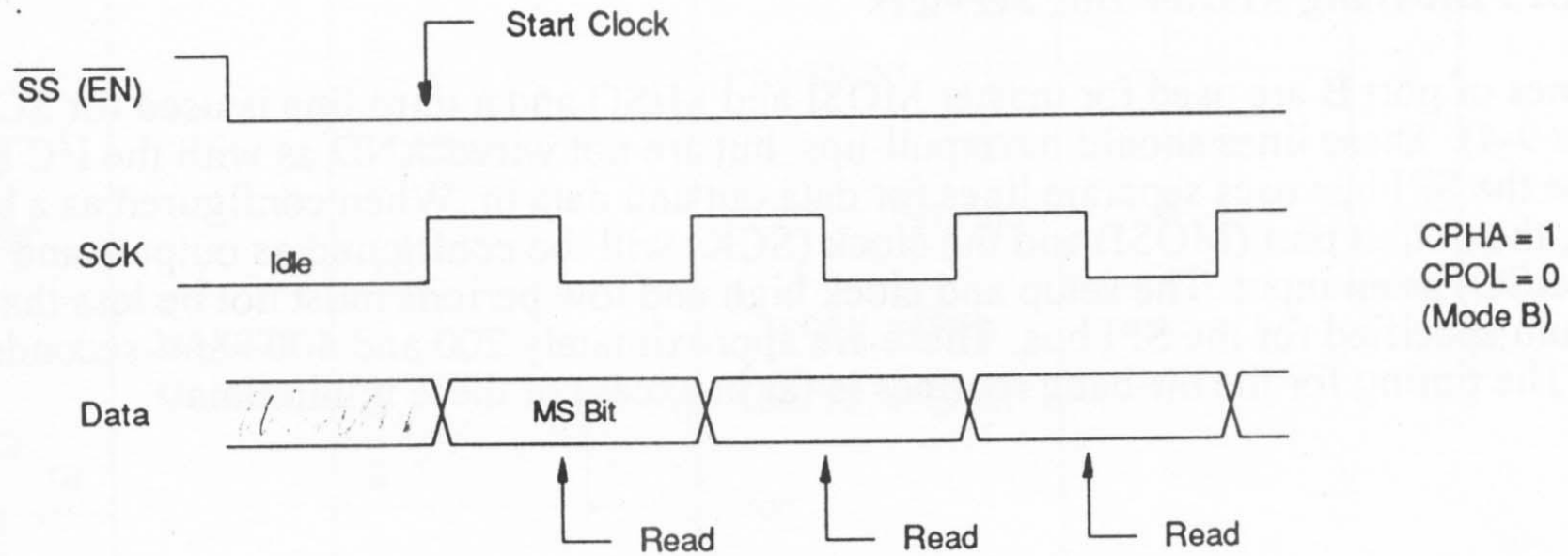


Figure 9-2 Typical SPI Bus Application

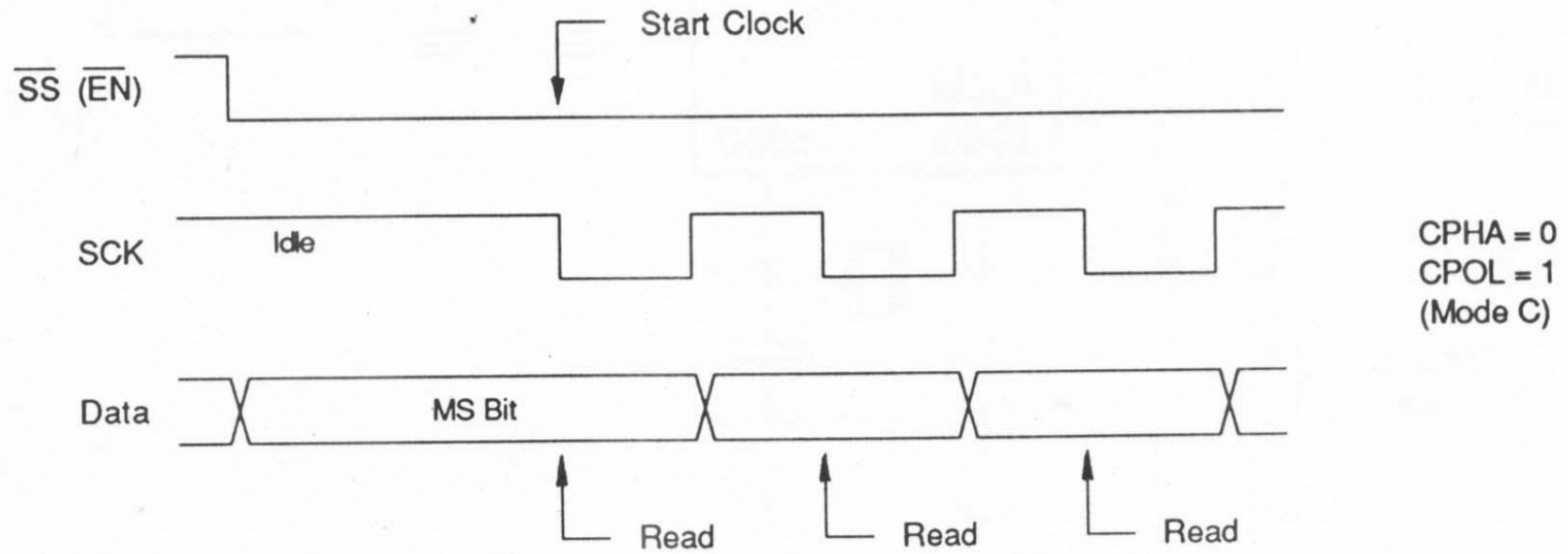
SPI SERIAL BUS – MODE A



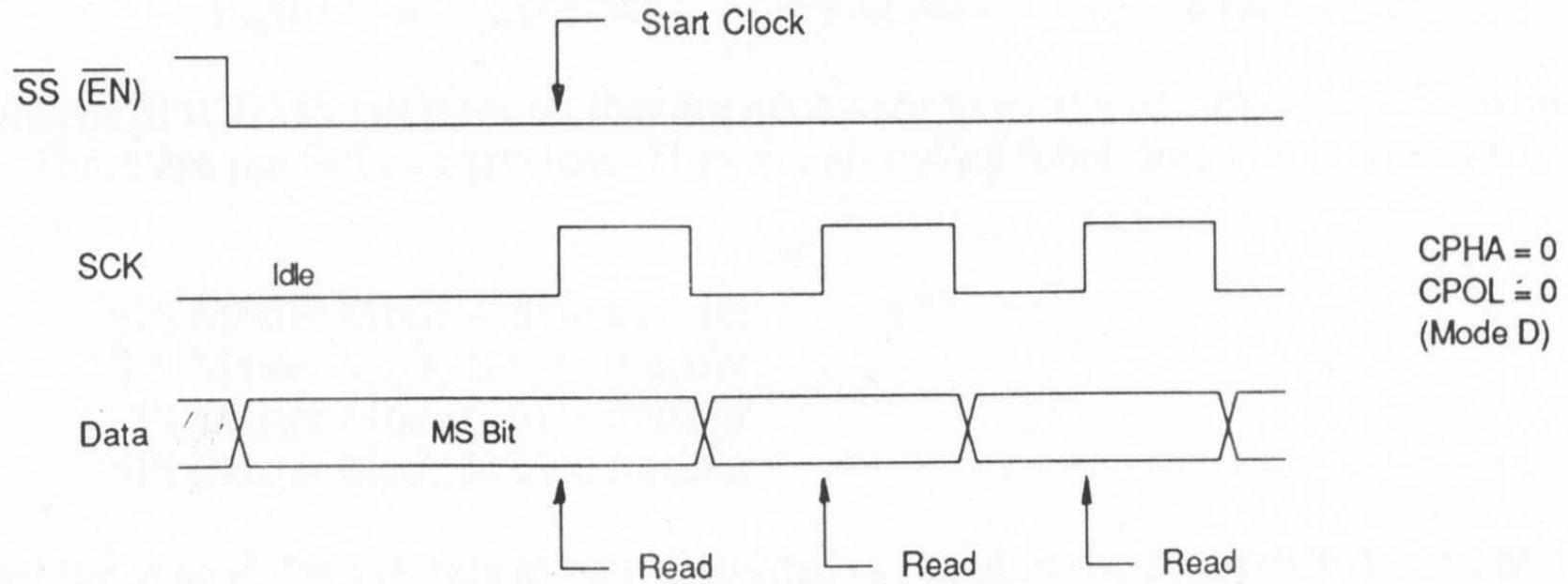
SPI SERIAL BUS – MODE B



SPI SERIAL BUS MODE C



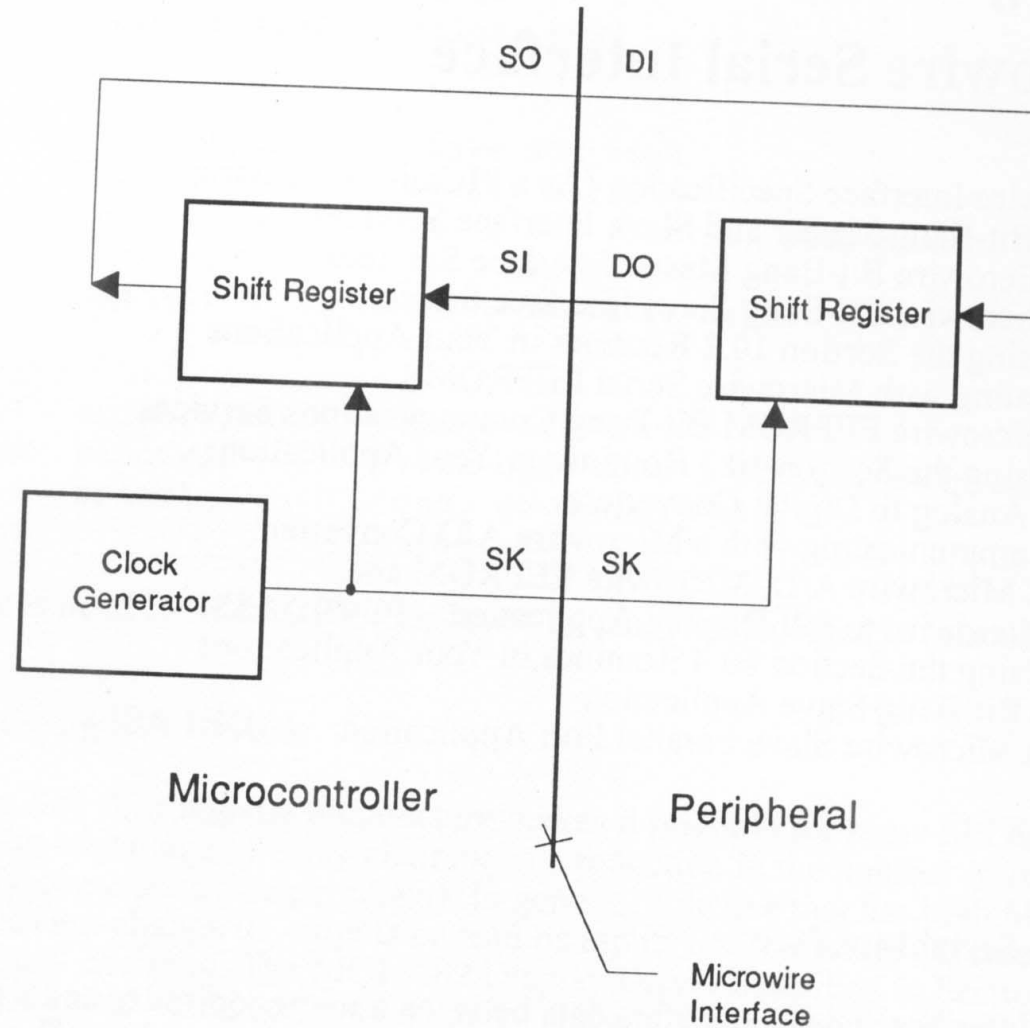
SPI SERIAL BUS – MODE D



MICROWIRE SERIAL INTERFACE

- **Cada dispositivo pode atuar simultaneamente como transmissor e receptor. O microcontrolador e cada dispositivo periférico tem um pino de saída, um de entrada e um pino de clock.**
- **Dispositivos se comunicando via microwire são denominados microcontroladores e periféricos.**
- **Cada dispositivo periférico é controlado por uma linha de “chip select”. Somente um periférico pode estar ativo de cada vez e uma seleção dedicada para cada periférico é necessária.**
- **A transferência microwire emprega um esquema simples de registrador de deslocamento, conforme figura a seguir**

MICROWIRE SERIAL INTERFACE



MICROWIRE SERIAL INTERFACE – ESPECIFICAÇÕES

- Os pinos de entrada e saída do mestre são designados **SO** (serial out) e **SI** (serial in), respectivamente.
- Os pinos de entrada e saída do periférico são designados **DO** (data out) e **DI** (data in), respectivamente.
- A saída do pino de seleção do mestre é denominada **PX** (peripheral exchange) e a entrada do pino de seleção no escravo é denominado **CS** (chip select).

MICROWIRE SERIAL INTERFACE

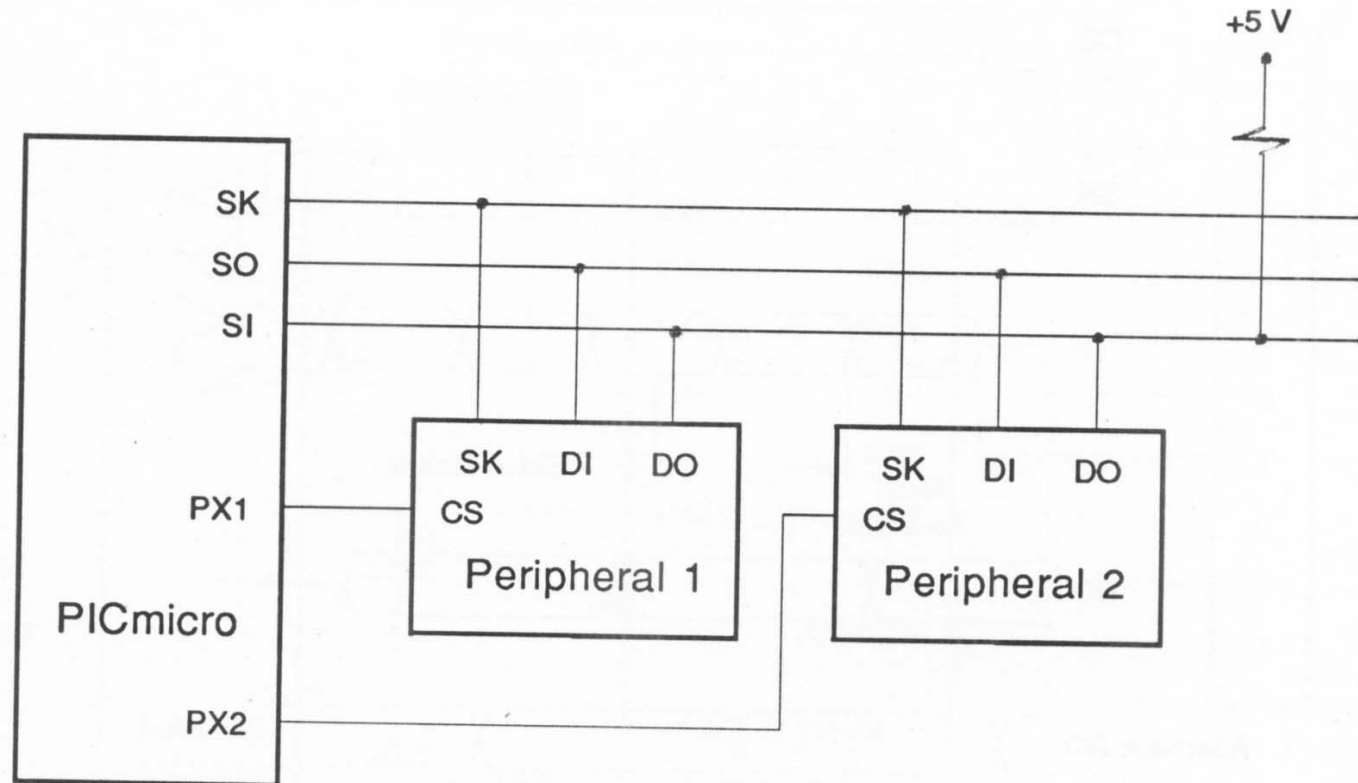
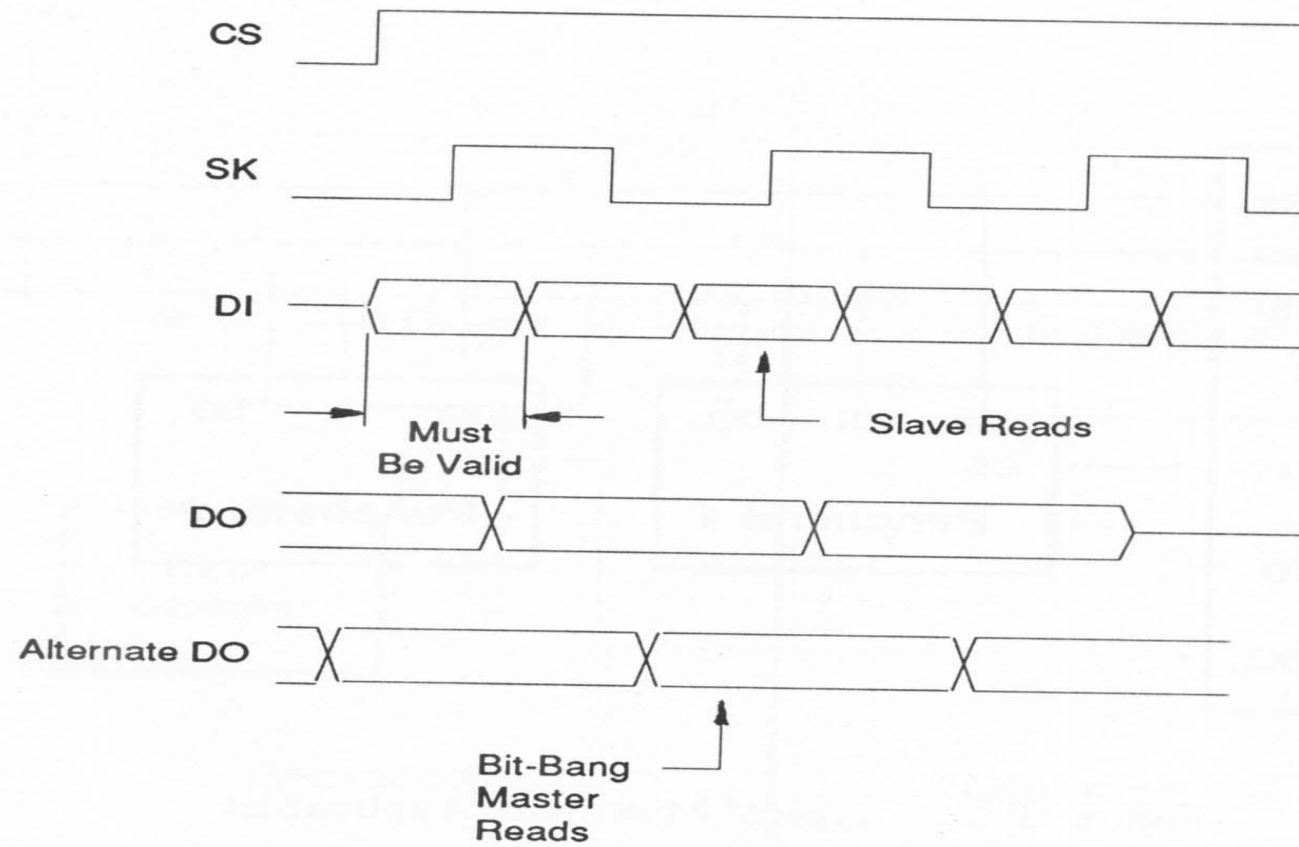
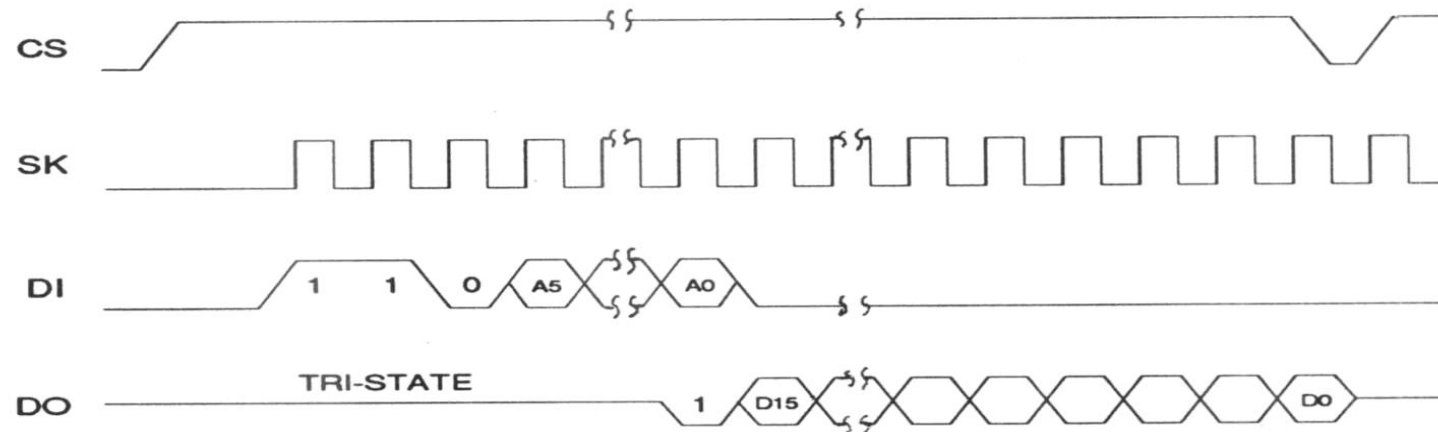


Figure 10-2 Microwire Serial Interface

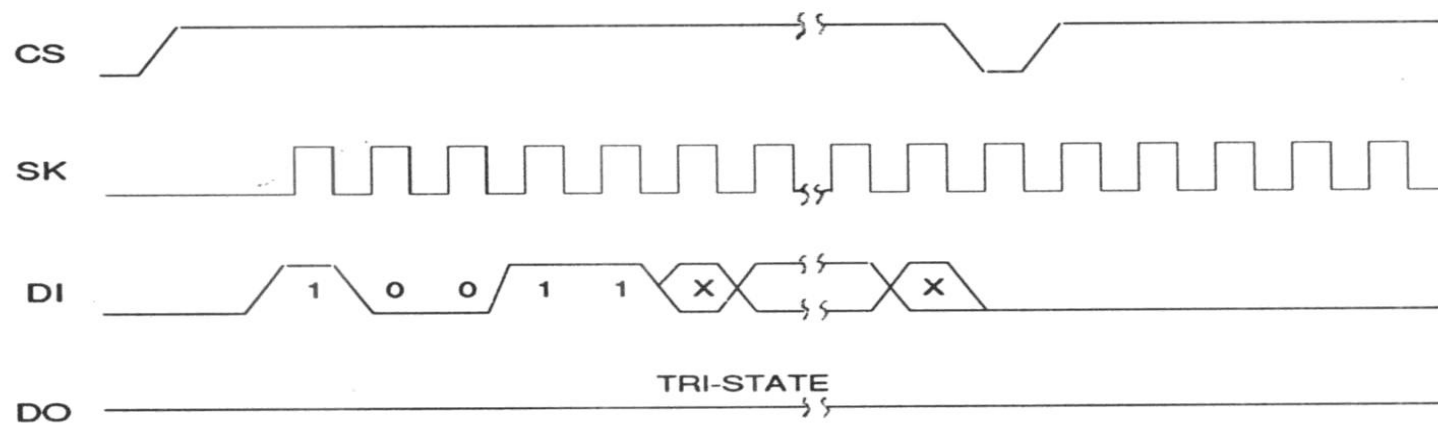
MICROWIDE SERIAL INTERFACE TIMING



MICROWIRE SERIAL INTERFACE – COMUNICAÇÃO COM EEPROM'S

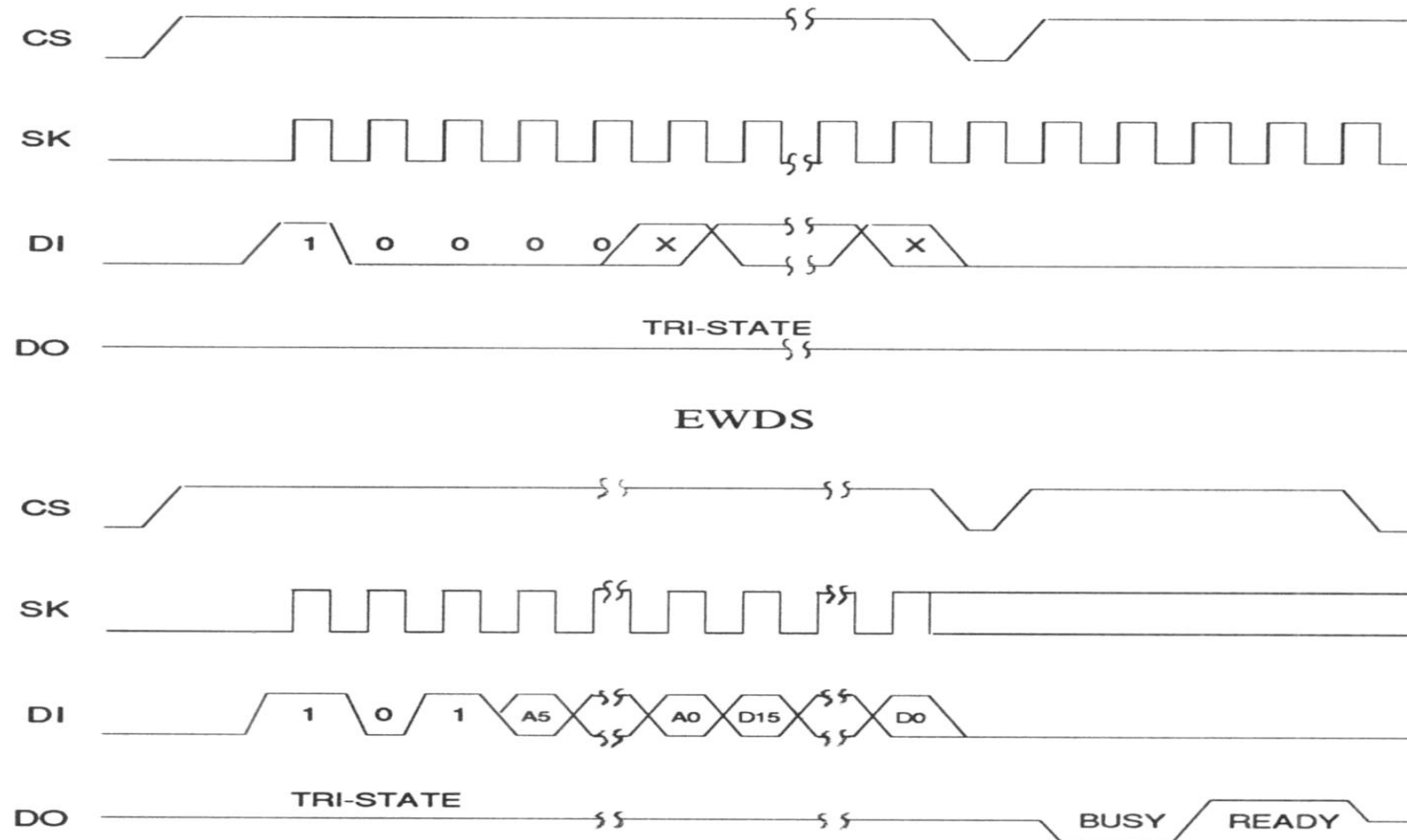


Read



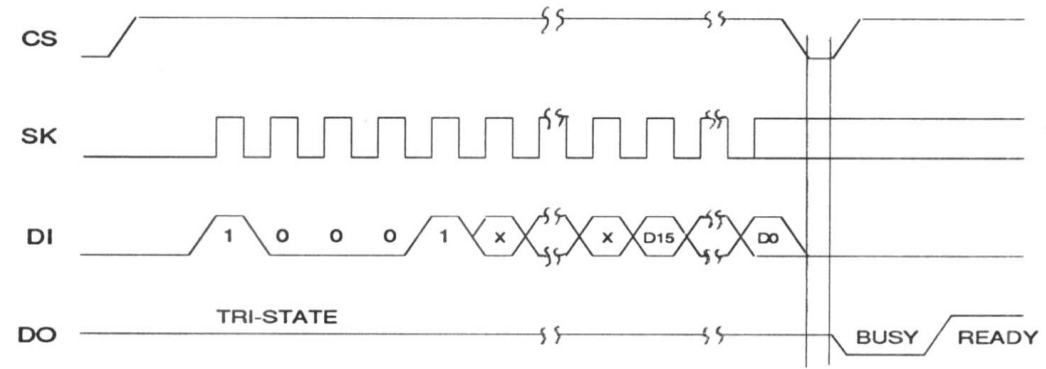
EWEN

MICROWIRE SERIAL INTERFACE – COMUNICAÇÃO COM EEPROM'S

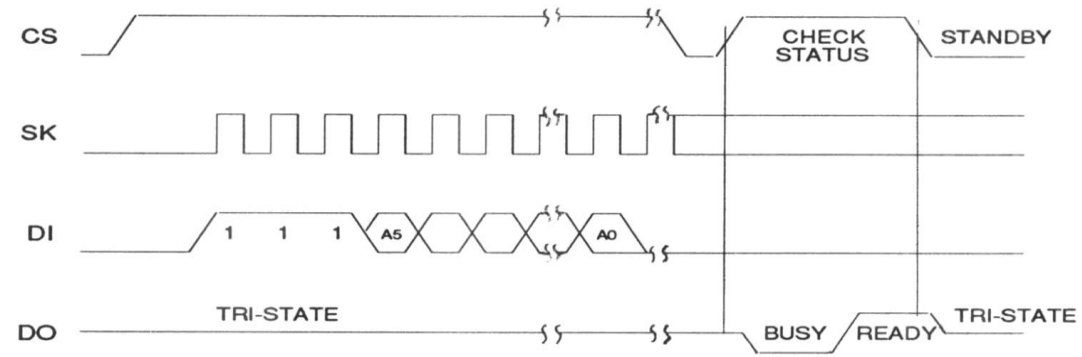


Write

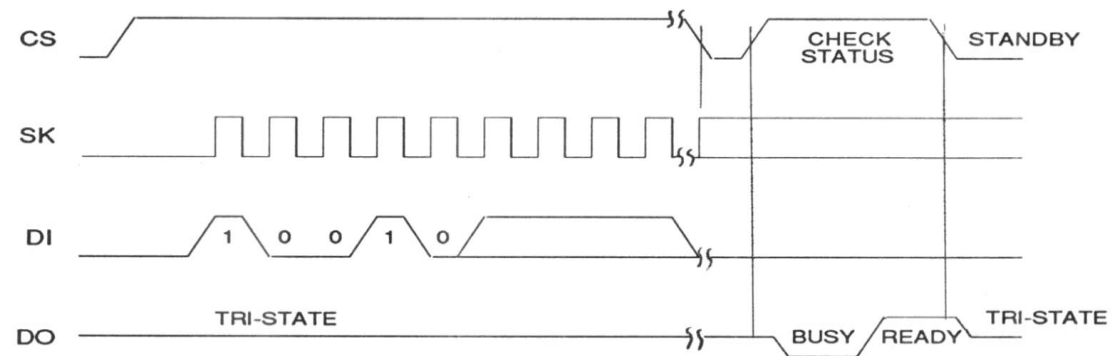
MICROWIRE SERIAL INTERFACE – COMUNICAÇÃO COM EEPROM'S



WRAL



Erase



ERAL

INTERFACE I2C STM32F103C8

REGISTRADORES RELACIONADOS

Register name	Offset	Description
I2C_CR1	0x00	I2C Control Register 1
I2C_CR2	0x04	I2C Control Register 2
I2C_OAR1	0x08	I2C Own Address Register 1
I2C_OAR2	0x0C	I2C Own Address Register 2
I2C_DR	0x10	I2C Data Register
I2C_SR1	0x14	I2C Status Register 1
I2C_SR2	0x18	I2C Status Register 2
I2C_CCR	0x1C	Clock Control Register
I2C_TRISE	0x20	I2C Trise

I2C_CR1

	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
I2Cx_CR1:	SW RST	Res.	ALERT	PEC	POS	ACK	STOP	START	NOST RETCH	ENGCG	ENPEC	EN ARP	SMB TYPE	Res.	SMBU S	PE

Field	Bit	Descriptions
SWRST	15	Software Reset
ALERT	13	SMBus alert (It is used in SMBus.)
PEC	12	Packet Error Checking
POS	11	PEC Position
ACK	10	Acknowledge enable (0: No Ack, 1: ACK) The bit is set/cleared by software to send ACK/NACK. Hardware clears the bit after sending ACK.
STOP	9	Stop generation If software sets the bit, the I2C hardware generates a Stop condition and switches to slave mode (The MSL bit of I2C_SR2 is cleared.)
START	8	Start generation When software sets the bit, the I2C interface generates a Start condition and switches to master mode (MSL is set). If it is already in master mode, setting the START bit generates a repeated start.
NOSTRETCH	7	No clock stretching in slave mode (0: clock stretching enabled, 1: disabled) Software can set the bit to disable clock stretching.
ENGCG	6	General Call Enable (0: General call disabled, 1: enabled (Addr. 0 is ACKed))
ENPEC	5	PEC enable (0: PEC calculation disabled, 1: PEC calculation enabled)
ENARP	4	ARP enable (It is used in SMBus.)
SMBTYPE	3	SMB Type (It is used in SMBus.)
SMBUS	1	SMB Bus (0: I2C mode, 1: SMBus mode)
PE	0	Peripheral Enable (0: peripheral disable, 1: enable)

I2C_CR2

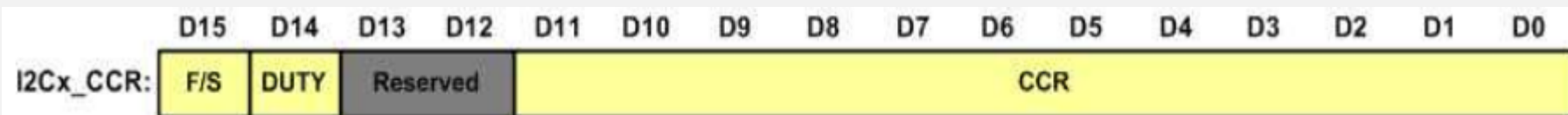
	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
I2Cx_CR2:	Reserved			LAST	DMA EN	ITBUF EN	ITEVT EN	ITERR EN	Reserved		FREQ					

Field	Bit	Descriptions
LAST	12	DMA Last Transfer
DMAEN	11	DMA request Enable (0: disabled, 1: enabled)
ITBUFEN	10	Buffer Interrupt Enable (0: Interrupt disabled, 1: enabled) If ITBUFEN is set, an interrupt is generated when TxE or RxNE flags of I2C_SR1 are set.
ITEVTEN	9	Event Interrupt Enable (0: Interrupt disabled, 1: enabled) If ITEVTEN is set, an interrupt is generated when any of the event flags (SB, ADDR, ADD10, STOPF, or BTF) are set.
ITERREN	8	Error Interrupt Enable (0: Interrupt disabled, 1: enabled) If ITERREN is set, an interrupt is generated when any of the error flags (BERR, ARLO, AF, OVR, PECERR, TIMEOUT, or SMBALERT) are set.
FREQ	5-0	PCLK (Peripheral Clock) Frequency

I2C_CCR

Velocidades usadas no padrão I2C

I2C grade	Baud rate
Standard	100Kbps
Fast	Up to 400Kbps
Fast plus	Up to 1Mbps
High Speed	Up to 3.2Mbps



Field	Bit	Descriptions
F/S	15	Master mode selection (0: Standard mode, 1: Fast mode)
DUTY	14	SCL clock duty cycle in Fast mode
CCR	11-0	Clock Control in master mode

F/S	DUTY	Duty cycle for SCL	t_{low}	t_{high}	T_{I2C} ($=t_{low}+t_{high}$)	Baud rate ($1/T_{I2C}$)
0 (Standard)	X	50%	$CCR \times T_{PCLK}$	$CCR \times T_{PCLK}$	$2 \times CCR \times T_{PCLK}$	$F_{PCLK}/(2 \times CCR)$
1 (Fast)	0	33.3%	$2 \times CCR \times T_{PCLK}$	$CCR \times T_{PCLK}$	$3 \times CCR \times T_{PCLK}$	$F_{PCLK}/(3 \times CCR)$
1 (Fast)	1	36%	$16 \times CCR \times T_{PCLK}$	$9 \times CCR \times T_{PCLK}$	$25 \times CCR \times T_{PCLK}$	$F_{PCLK}/(25 \times CCR)$

I2C_TRISE

- configura o tempo de subida (rise time) do sinal I2C. Esse ajuste é necessário para que o periférico I2C atenda às especificações do protocolo I2C da NXP/Philips, que impõe limites ao tempo de subida das linhas SDA e SCL, de acordo com a frequência de operação.
- De acordo com o manual, o valor de configuração do registro TRISE para o modo padrão (100KHz)

$$\text{TRISE} = (\text{maximum_rise_time} / T_{\text{I2CCLK}}) + 1$$

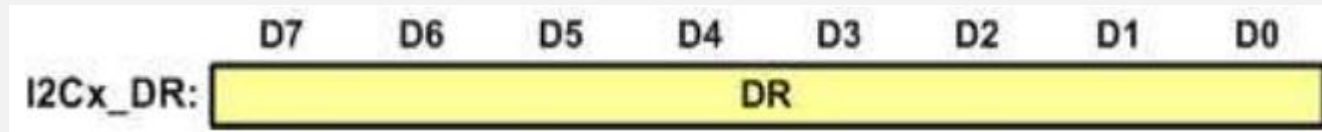
maximum_rise_time:

- 1000ns para modo padrão (Standard Mode, 100 kHz)
- 300ns para modo rápido (Fast Mode, 400 kHz)
- $T_{\text{I2CCLK}} = 1 / \text{frequência do periférico I2C (geralmente PCLK1)}$
- Exemplo:
Suponha:
 - $\text{PCLK1} = 36 \text{ MHz} \rightarrow T_{\text{I2CCLK}} = 1 / 36\text{MHz} = 27.77\text{ns}$
 - Modo padrão (100kHz) $\rightarrow \text{maximum_rise_time} = 1000\text{ns}$

$$\text{TRISE} = (1000\text{ns} / 27.77\text{ns}) + 1 \approx 36$$

I2C_DR

- Registro que armazena o dado a ser transmitido (escrita) e também a ser recebido (leitura).

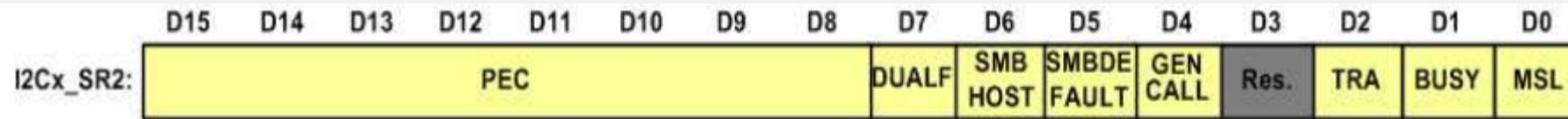


I2C_SR1 E I2C_SR2 - REGISTRO DE STATUS

	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
I2Cx_SR1:	SMB ALERT	TIME OUT	Res.	PEC ERR	OVR	AF	ARLO	BERR	TxE	RxNE	Res.	STOPF	ADD10	BTF	ADDR	SB

Field	Bit	Descriptions
SMBALERT	15	SMB Alert (Not used in I2C)
TIMEOUT	14	Time out (0: No timeout, 1: SCL remained LOW for 25ms)
PECERR	12	PEC Error in reception
OVR	11	Overflow/Underflow (0: No overflow/underflow, 1: overflow or underflow) The flag might rise when NOSTRETCH=1. See the reference manual.
AF	10	Acknowledge Failure (0: No Acknowledge failure, 1: Acknowledge failure)
ARLO	9	Arbitration lost in master mode (0: no arbitration lost, 1: arbitration lost)
BERR	8	Bus Error (0: no bus error, 1: bus error) As discussed earlier, when SCL is high, SDA should not change. Otherwise, it is considered as STOP or START conditions. While transferring data, if the value of SDA changes when SCL is high, the BERR flag sets.
TxE	7	Transmit Empty (0: Transmit not empty, 1: Empty) The flag is set by hardware when it is waiting for us to write data to the I2C_DR register.
RxNE	6	Receive Not Empty The flag is set when a new byte of data is in I2C_DR waiting to be read.
STOPF	4	Stop detection in slave mode (0: No Stop condition, 1: Stop condition)
ADD10	3	10-bit header sent (Used in 10-bit address mode)
BTF	2	Byte transfer finished (0: transfer not done, 1: transfer successfully finished)
ADDR	1	Address sent (master mode)/matched (slave mode) In slave mode, if the received address matches with I2C_OAR, the flag sets. In master mode, when the address is sent, the flag sets. To clear the flag, read I2C_SR1 and then I2C_SR2.
SB	0	Start bit (0: No start generated, 1: start condition generated) In master mode, the flag sets as soon as a Start condition is generated. To clear the flag, read I2C_SR1 and then write to I2C_DR.

I2C_SR1 E I2C_SR2 - REGISTRO DE STATUS



Field	Bit	Descriptions
PEC	15-8	Packet Error Checking (For more information, see the manual.)
DUALF	7	Dual Flag (It is used when dual address is enabled. See the manual.)
SMBHOST	6	Used in SMBus. (For more information, see the manual.)
SMBDEFAULT	5	Used in SMBus. (For more information, see the manual.)
GENCALL	4	General call detected (0: no general call, 1: General call address received)
TRA	2	Transmitter/receiver (0: receiver, 1: transmitter) In slave mode, the flag shows if the device is in receiver or transmitter mode. The hardware sets the flag according to the R/W signal.
BUSY	1	Bus Busv (0: no communication on the bus, 1: communication on the bus)
		The flag indicates that a communication is in progress. Hardware sets the flag when SCL or SDA become low. The flag clears when a stop condition is detected.
MSL	0	Master/Slave (0: Slave, 1: Master) The hardware sets the flag when the I2C module is in master mode. The flag is set when a stop condition or arbitration lost is detected.

I2C_SR1 E I2C_SR2 – REGISTRO DE STATUS

Configurando o I2C

Precisamos seguir os seguintes passos para configurar o I2C:

1. Habilitar o clock para o módulo I2C e para o GPIO usando os registradores APB1ENR e APB2ENR.
2. Configurar os pinos do I2C como saída de função alternativa em modo open-drain.
3. Inicializar o campo FREQ do registrador CR2 com a frequência de PCLK.
4. Inicializar o registrador CCR para definir corretamente a taxa de transmissão em modo mestre.
5. Inicializar o registrador TRISE.
6. Habilitar o módulo I2C configurando o bit PE do registrador CCR1.

I2C_SR1 E I2C_SR2 – REGISTRO DE STATUS

Enviando dados no modo mestre

Para enviar dados no modo mestre, devemos fazer o seguinte:

1. Verificar o bit de ocupação do barramento (busy flag) no registrador SR2 para garantir que o barramento não está ocupado.
2. Configurar o bit START do registrador CR1 para gerar uma condição de início.
3. Monitorar o bit SB do registrador SR1 até que a condição de início seja gerada.
4. Colocar o endereço do escravo no registrador DR (Data Register). Os bits de 1 a 7 devem conter o endereço do escravo e o bit 0 indica R/W. Para enviar dados, o bit R/W deve ser 0.
5. Monitorar os registradores de status. Se o endereço for enviado com sucesso, o bit ADDR será setado e você pode continuar. Se o bit ARLO (Perda de Arbitragem) for setado, aguarde até que o barramento esteja livre e repita os passos de 1 a 5.
6. Carregar o registrador de dados com o byte a ser enviado.
7. Monitorar o bit TxE. Este bit é setado quando um ACK é recebido.
8. Repetir os passos 6 e 7 se houver mais dados a serem enviados. Caso contrário, configurar o bit STOP do registrador CR1 para gerar uma condição de parada.

I2C_SR1 E I2C_SR2 – REGISTRO DE STATUS

Recebendo dados no modo mestre

Receber dados é similar ao envio. Para receber dados no modo mestre, devemos fazer o seguinte:

1. Verificar o bit de ocupação do barramento (busy flag) no registrador SR2 para garantir que o barramento não está ocupado.
2. Configurar o bit START do registrador CR1 para gerar uma condição de início.
3. Monitorar o bit SB do registrador SR1 até que a condição de início seja gerada.
4. Colocar o endereço do escravo no registrador DR. Os bits de 1 a 7 devem conter o endereço do escravo e o bit 0 indica R/W. Para receber dados, o bit R/W deve ser 1.
5. Monitorar os registradores de status. Se o endereço for enviado com sucesso, o bit ADDR será setado e você pode continuar. Se o bit ARLO (Perda de Arbitragem) for setado, aguarde até que o barramento esteja livre e repita os passos de 1 a 5.
6. Se quiser enviar um ACK em resposta, configure o bit ACK do registrador CR1.
7. Monitorar o bit RxNE. Este bit é setado quando um byte é recebido. Então, leia o registrador de dados para obter o byte recebido.
8. Repetir os passos 6 e 7 se quiser receber mais bytes. Caso contrário, configurar o bit STOP do registrador CR1 para gerar uma condição de parada.

FUNÇÕES DISPONÍVEIS NA BIBLIOTECA HAL

Funções síncronas (bloqueantes) — sempre disponíveis:

Função

HAL_I2C_Master_Transmit

HAL_I2C_Master_Receive

HAL_I2C_Mem_Write

HAL_I2C_Mem_Read

HAL_I2C_IsDeviceReady

HAL_I2C_GetState

HAL_I2C_GetError

Descrição

Envia dados diretamente a um escravo

Recebe dados diretamente de um escravo

Escreve em registrador interno

Lê de registrador interno

Verifica se o escravo está respondendo

Retorna o estado atual do I2C

Retorna o código de erro da última operação